

Relational Algebra

Relational database systems are expected to be equipped with a query language that can assist its users to query the database instances. There are two kinds of query languages – relational algebra and relational calculus.

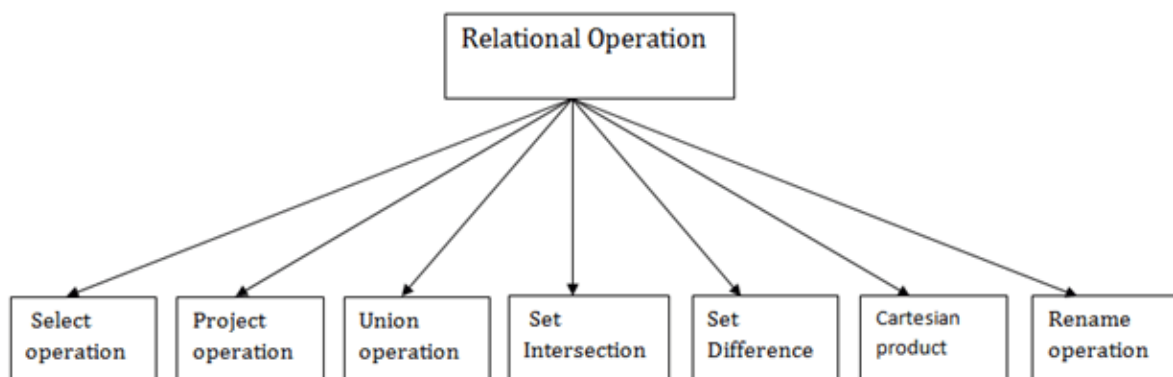
Relational Algebra

Relational algebra is a procedural query language, which takes instances of relations as input and yields instances of relations as output. It uses operators to perform queries. An operator can be either **unary** or **binary**. They accept relations as their input and yield relations as their output. Relational algebra is performed recursively on a relation and intermediate results are also considered relations.

The fundamental operations of relational algebra are as follows –

- ✓ Select
 - ✓ Project
 - ~~Union~~
 - ~~Set different~~
 - ~~Cartesian product~~
 - ✓ Rename
- unary*
-] Binary → Intersection (∩)*

Types of Relational operation



Select Operation (σ)

It selects tuples that satisfy the given predicate from a relation.

Notation – $\sigma_p(r)$

Where σ stands for selection predicate and r stands for relation. p is propositional logic formula which may use connectors like **and**, **or**, and **not**. These terms may use relational operators like $=$, $\neq$, $\geq$, $<$, $>$, $\leq$.

For example –

$\sigma_{subject = "database"}(\text{Books})$

Output – Selects tuples from books where subject is 'database'.

$\sigma_{subject = "database" \text{ and } price = "450"}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450.

$\sigma_{subject = "database" \text{ and } price = "450" \text{ or } year > "2010"}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450 or those books published after 2010.

For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Mianus	L-13	500
Roundhill	L-11	900
Perryride	L-16	1300

Input:

1. $\sigma_{\text{BRANCH_NAME} = "perryride"}(\text{LOAN})$

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

Project Operation (Π)

It projects column(s) that satisfy a given predicate.

Notation – $\Pi_{A_1, A_2, A_n}(r)$

Where A_1, A_2, A_n are attribute names of relation r .

Duplicate rows are automatically eliminated, as relation is a set.

For example –

$\Pi_{\text{subject, author}}(\text{Books})$

Selects and projects columns named as subject and author from the relation Books.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye
Johnson	Alma	Brooklyn
Brooks	Senator	Brooklyn

Input:

1. $\Pi_{\text{NAME, CITY}}(\text{CUSTOMER})$

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye
Johnson	Brooklyn

Brooks	Brooklyn
--------	----------

Union Operation (U)

It performs binary union between two given relations and is defined as –

$$r \cup s = \{ t \mid t \in r \text{ or } t \in s \}$$

Notation – $r \cup s$

Where **r** and **s** are either database relations or relation result set (temporary relation).

For a union operation to be valid, the following conditions must hold –

- **r**, and **s** must have the same number of attributes.
- Attribute domains must be compatible.
- Duplicate tuples are automatically eliminated.

$$\Pi_{\text{author}}(\text{Books}) \cup \Pi_{\text{author}}(\text{Articles})$$

Output – Projects the names of the authors who have either written a book or an article or both.

Example:

DEPOSITOR RELATION

CUSTOMER_NAME	ACCOUNT_NO
Johnson	A-101
Smith	A-121
Mayes	A-321
Turner	A-176
Johnson	A-273
Jones	A-472
Lindsay	A-284

BORROW RELATION

CUSTOMER_NAME	LOAN_NO
Jones	L-17
Smith	L-23
Hayes	L-15
Jackson	L-14
Curry	L-93
Smith	L-11
Williams	L-17

Input:

1. $\prod \text{CUSTOMER_NAME (BORROW)} \cup \prod \text{CUSTOMER_NAME (DEPOSITOR)}$

Output:

CUSTOMER_NAME
Johnson
Smith
Hayes
Turner
Jones
Lindsay

Jackson
Curry
Williams
Mayes

Set Intersection:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection $\cap$.

1. Notation: $R \cap S$

Example: Using the above DEPOSITOR table and BORROW table

Input:

1. $\Pi_{\text{CUSTOMER_NAME}}(\text{BORROW}) \cap \Pi_{\text{CUSTOMER_NAME}}(\text{DEPOSITOR})$

Output:

CUSTOMER_NAME
Smith
Jones

Set Difference ($-$)

The result of set difference operation is tuples, which are present in one relation but are not in the second relation.

Notation – $r - s$

Finds all the tuples that are present in **r** but not in **s**.

$\Pi_{\text{author}}(\text{Books}) - \Pi_{\text{author}}(\text{Articles})$

Output – Provides the name of authors who have written books but not articles.

Cartesian Product (X)

Combines information of two different relations into one.

Notation – $r \times s$

Where **r** and **s** are relations and their output will be defined as –

$$r \times s = \{ q \mid q \in r \text{ and } t \in s \}$$

$\sigma_{\text{author} = \text{'tutorialspoint'}}(\text{Books} \times \text{Articles})$

Output – Yields a relation, which shows all the books and articles.

Example:

A	B
(Name Age Sex)	(Id Course)
-----	-----
Ram 14 M	1 DS
Sona 15 F	2 DBMS
kim 20 M	

$A \times B$

Name	Age	Sex	Id	Course

Ram	14	M	1	DS
Ram	14	M	2	DBMS
Sona	15	F	1	DS
Sona	15	F	2	DBMS
Kim	20	M	1	DS
Kim	20	M	2	DBMS

Rename Operation (ρ)

The results of relational algebra are also relations but without any name. The rename operation allows us to rename the output relation. 'rename' operation is denoted with small Greek letter **rho** ρ .

Notation – $\rho_x(E)$

Where the result of expression **E** is saved with name of **x**.

Join Operations:

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by $\bowtie$.

Example:

EMPLOYEE

EMP_CODE	EMP_NAME
101	Stephan
102	Jack
103	Harry

SALARY

EMP_CODE	SALARY
101	50000
102	30000
103	25000

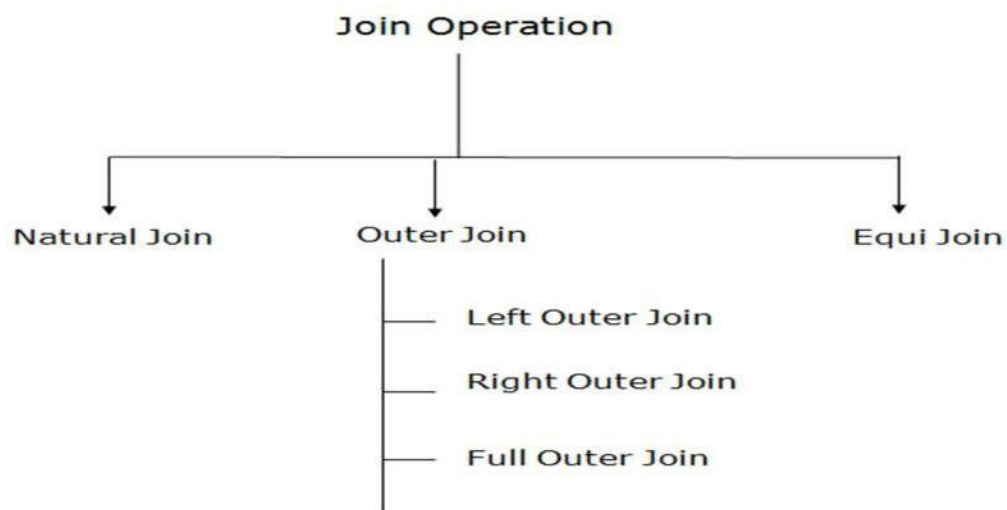
Operation: (EMPLOYEE $\bowtie$ SALARY)

Result:

EMP_CODE	EMP_NAME	SALARY
----------	----------	--------

101	Stephan	50000
102	Jack	30000
103	Harry	25000

Types of Join operations:



1. Natural Join:

- A natural join is the set of tuples of all combinations in R and S that are equal on their common attribute names.
- It is denoted by $\bowtie$.

Example: Let's use the above EMPLOYEE table and SALARY table:

Input:

$\bowtie$ EMP_NAME, SALARY (EMPLOYEE $\bowtie$ SALARY)

Output:

EMP_NAME	SALARY
Stephan	50000
Jack	30000
Harry	25000

2. Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missing information.

Example:

EMPLOYEE

EMP_NAME	STREET	CITY
Ram	Civil line	Mumbai
Shyam	Park street	Kolkata
Ravi	M.G. Street	Delhi
Hari	Nehru nagar	Hyderabad

FACT_WORKERS

EMP_NAME	BRANCH	SALARY
Ram	Infosys	10000

Shyam	Wipro	20000
Kuber	HCL	30000
Hari	TCS	50000

An outer join is basically of three types:

- a. Left outer join
- b. Right outer join
- c. Full outer join

a. Left outer join:

- Left outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

EMPLOYEE $\bowtie$ FACT_WORKERS

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL

b. Right outer join:

- Right outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.

- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS Relation

Input:

EMPLOYEE $\bowtie$ FACT_WORKERS

Output:

EMP_NAME	BRANCH	SALARY	STREET	CITY
Ram	Infosys	10000	Civil line	Mumbai
Shyam	Wipro	20000	Park street	Kolkata
Hari	TCS	50000	Nehru street	Hyderabad
Kuber	HCL	30000	NULL	NULL

C. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from both tables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S that have no matching tuples in R in their common attribute name.
- It is denoted by $\bowtie$.

Example: Using the above EMPLOYEE table and FACT_WORKERS table

Input:

EMPLOYEE $\bowtie$ FACT_WORKERS

Output:

EMP_NAME	STREET	CITY	BRANCH	SALARY
Ram	Civil line	Mumbai	Infosys	10000
Shyam	Park street	Kolkata	Wipro	20000
Hari	Nehru street	Hyderabad	TCS	50000
Ravi	M.G. Street	Delhi	NULL	NULL
Kuber	NULL	NULL	HCL	30000

3. Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data as per the equality condition. The equi join uses the comparison operator(=).

Example:

Π EMP_NAME (σ SALARY=10000) (EMPLOYEE $\bowtie$ FACT_WORKERS)

